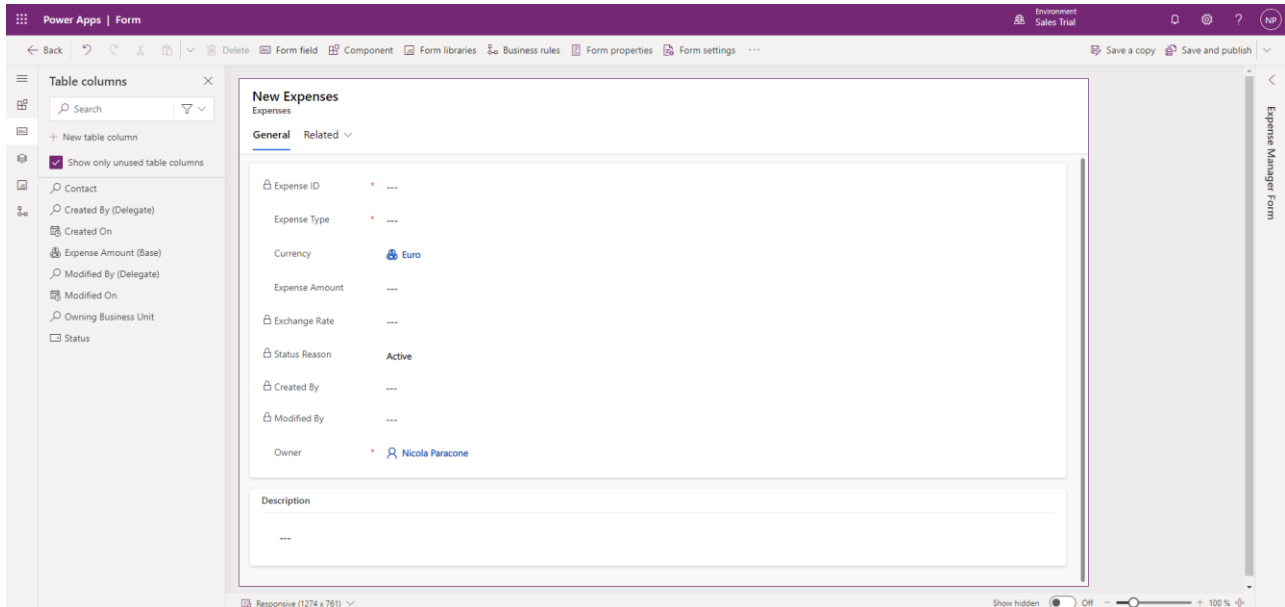


Implementation of JavaScript web resources

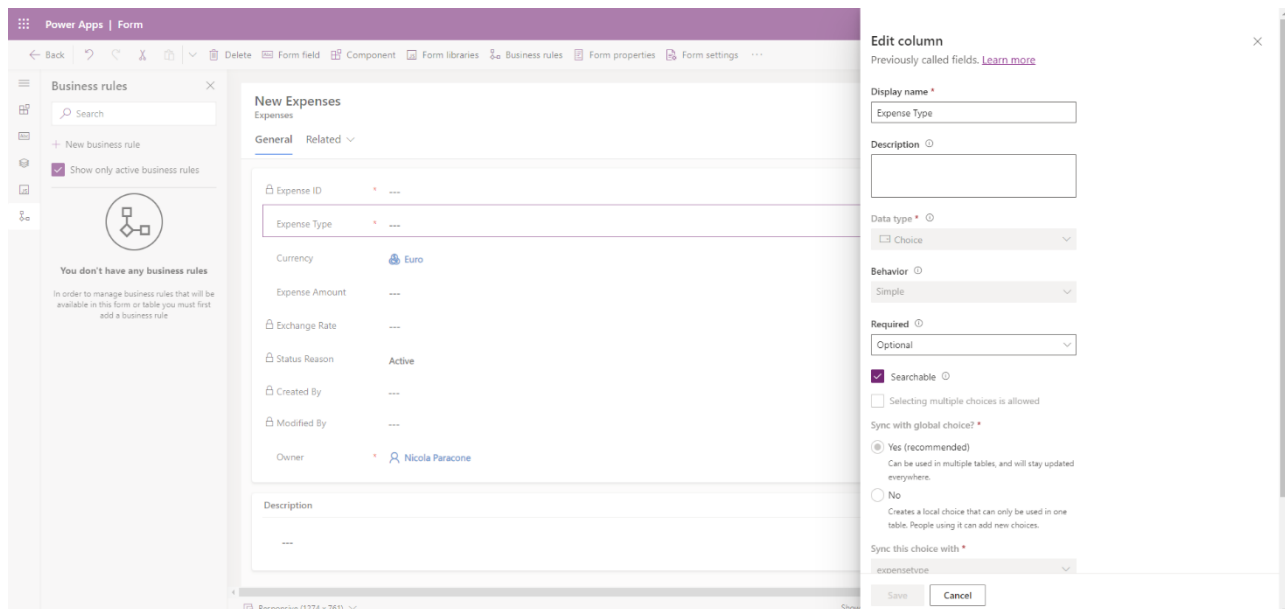
1. Make fields mandatory and hide/show fields dynamically

Let us take our Expense Manager Form.



In this lecture we write some JavaScript web resources to update the content of certain selected fields.

Turn off the business rules we have created and make sure to set to Optional the Required field of Expense Type.



The JavaScript web resource we want to create will set the Expense Amount as mandatory when Expense Type changes.

First, we write a script for displaying an alert when Expense Type is equal to Flight.

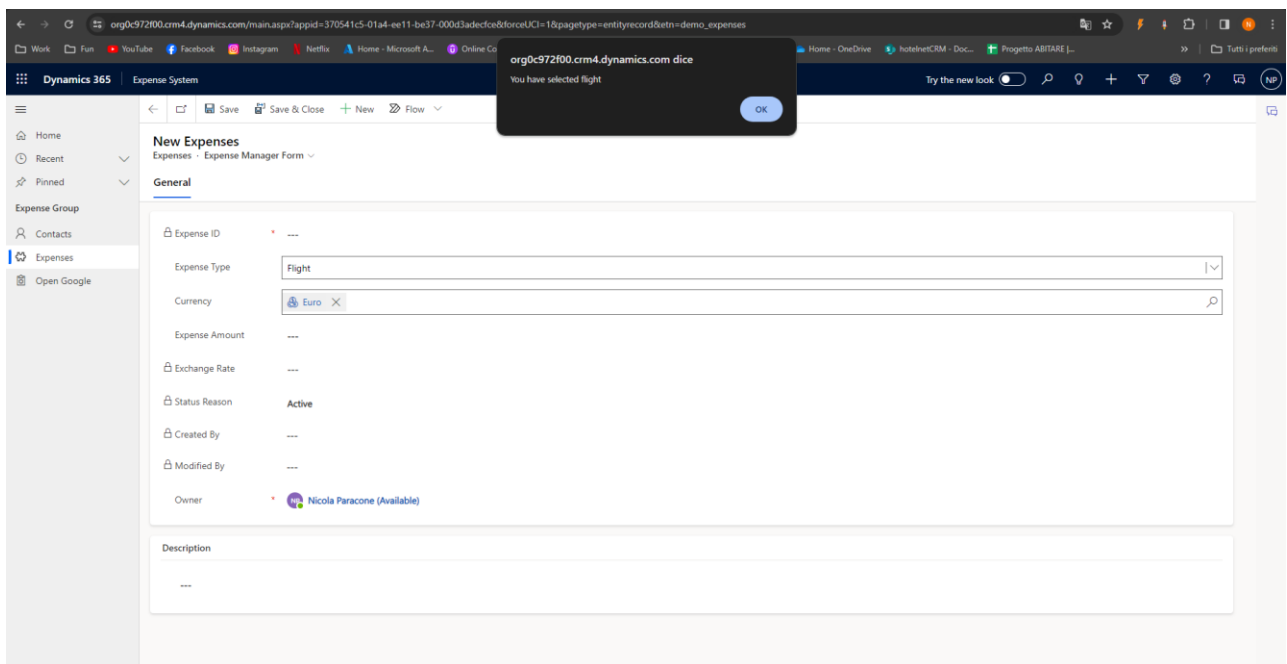
```
function onChangeOfTypeSetAmountMandatory(executionContext) {
    debugger;

    var formcontext = executionContext.getFormContext();
    // Acquire the value of Expense Type (not the label!)
    // Use Show Optionset Value in Power Pane or Level up

    var expenseType = formcontext.getAttribute('demo_expensetype').getValue();

    if (expenseType == 100000000) {
        alert("You have selected flight");
    }
}
```

The effect will be the following when we select Expense Type = Flight in the Expense Manager Form.



If you encounter problems, remember that you can always insert debugger statements and open the console in the browser to perform the debugging.

Now, we add to the script a statement which makes visible Expense Amount if Expense Type is Flight, otherwise it hides it.

```
function onChangeOfTypeSetAmountMandatory(executionContext){
    debugger;

    var formcontext = executionContext.getFormContext();
    // Acquire the value of Expense Type (not the label!)
    // Use Show Optionset Value in Power Pane or Level up

    var expenseType = formcontext.getAttribute('demo_expensetype').getValue();

    if (expenseType == 100000000){
        formcontext.getControl("demo_expenseamount").setVisible(true);
        alert("You have selected flighth expense");
    } else {
        formcontext.getControl("demo_expenseamount").setVisible(false);
    }
}
```

The different appearance of the form is the following.

This screenshot shows the 'New Expenses' form in Dynamics 365. The form is titled 'New Expenses - Unsaved' and 'Expenses - Expense Manager Form'. The 'General' tab is selected. The form contains the following fields:

- Expense ID: ---
- Expense Type: Flight
- Currency: Euro
- Expense Amount: ---
- Exchange Rate: ---
- Status Reason: Active
- Created By: ---
- Modified By: ---
- Owner: Nicola Paracone (Available)
- Description: ---

This screenshot shows the 'New Expenses' form in Dynamics 365. The form is titled 'New Expenses - Unsaved' and 'Expenses - Expense Manager Form'. The 'General' tab is selected. The form contains the following fields:

- Expense ID: ---
- Expense Type: Food
- Currency: Euro
- Exchange Rate: ---
- Status Reason: Active
- Created By: ---
- Modified By: ---
- Owner: Nicola Paracone (Available)
- Description: ---

Notice that in the second figure Expense Amount disappeared.

At this point, we insert the statements to make mandatory (or not) a field (Expense Amount) based on Expense Type being Flight. This is done by using `setRequiredLevel`.

The options supported by `setRequiredLevel` are required, none, recommended.

```
function onChangeOfTypeSetAmountMandatory(executionContext) {
    debugger;

    var formcontext = executionContext.getFormContext();
    // Acquire the value of Expense Type (not the label!)
    // Use Show Optionset Value in Power Pane or Level up

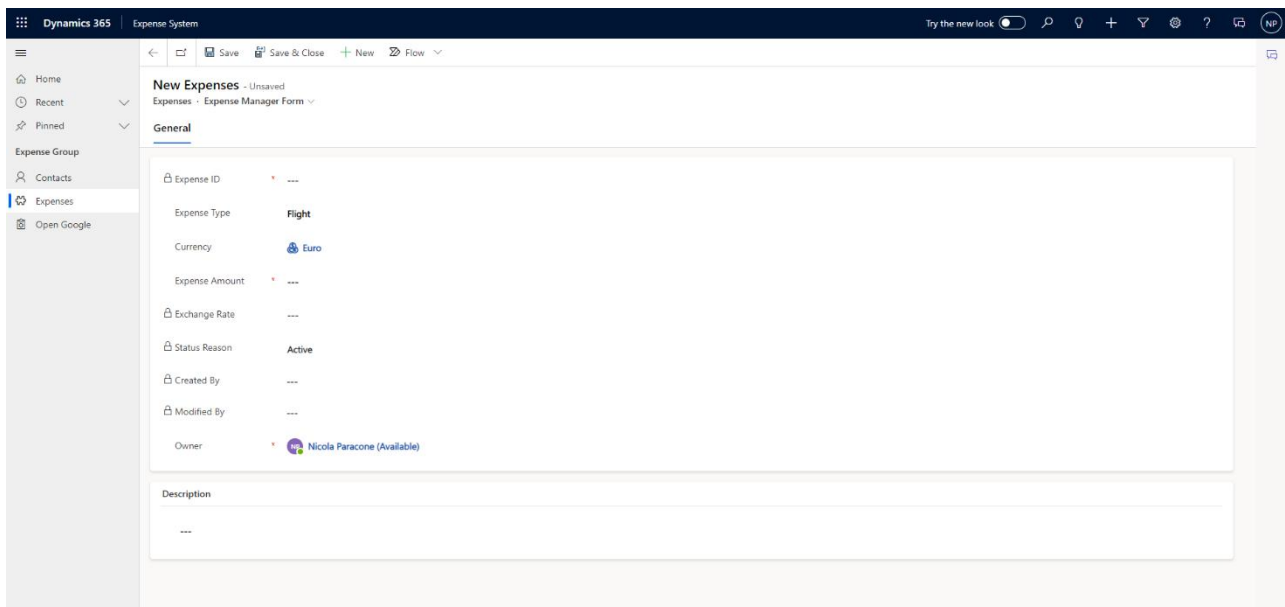
    var expenseType = formcontext.getAttribute('demo_expensetype').getValue();

    if (expenseType == 100000000) {

        formcontext.getControl("demo_expenseamount").setVisible(true);
        formcontext.getAttribute("demo_expenseamount").setRequiredLevel("required");

        alert("You have selected flight expense");
    } else {
        formcontext.getAttribute("demo_expenseamount").setRequiredLevel("none");
        formcontext.getControl("demo_expenseamount").setVisible(false);
    }
}
```

Notice the effect in Dynamics 365.



The screenshot shows the Dynamics 365 user interface for a 'New Expenses' form. The left-hand navigation pane includes links for Home, Recent, Pinned, Expense Group, Contacts, Expenses (which is currently selected), and Open Google. The main content area displays the 'New Expenses' form, which is titled 'Expenses - Expense Manager Form'. The 'General' tab is active, showing various fields: 'Expense ID' (empty), 'Expense Type' (set to 'Flight'), 'Currency' (set to 'Euro'), 'Expense Amount' (empty), 'Exchange Rate' (empty), 'Status Reason' (set to 'Active'), 'Created By' (empty), 'Modified By' (empty), and 'Owner' (set to 'Nicola Paracone (Available)'). Below these fields is a 'Description' section with a text area. The 'Expense Amount' field is visible and has a red asterisk, indicating it is required, which is the result of the JavaScript code shown in the previous block.

Navigate to <https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/walkthrough-write-your-first-client-script> to obtain more information about client-side scripting.

Let us add the Contact, Email and Yes or No fields to our Expense Manager Form.

The screenshot shows the Dynamics 365 interface for a 'New Expenses' form. The left sidebar contains navigation links for Home, Recent, Pinned, Expense Group, Contacts, Expenses (selected), and Open Google. The top bar shows 'Dynamics 365' and 'Expense System'. The form title is 'New Expenses' with a subtitle 'Expenses - Expense Manager Form'. The 'General' tab is selected. The form fields are as follows:

- Expense ID: Required field, currently empty.
- Expense Type: Dropdown menu, currently empty.
- Currency: Dropdown menu, set to 'Euro'.
- Expense Amount: Text input field, currently empty.
- Contact: Text input field, currently empty.
- Email: Text input field, currently empty.
- Yes or No: Radio buttons, 'No' is selected.
- Exchange Rate: Text input field, currently empty.
- Status Reason: Dropdown menu, set to 'Active'.
- Created By: Text input field, currently empty.
- Modified By: Text input field, currently empty.
- Owner: Text input field, set to 'Nicola Paracone'.
- Description: Text area, currently empty.

The following code, for example, contains the statement for managing the Yes or No field.

```
function onChangeOfTypeSetAmountMandatory(executionContext) {
    debugger;

    var formcontext = executionContext.getFormContext();
    // Acquire the value of Expense Type (not the label!)
    // Use Show Optionset Value in Power Pane or Level up

    var expenseType = formcontext.getAttribute('demo_expensetype').getValue();

    // Set the optionset value for Expense Type
    formcontext.getAttribute("demo_expensetype").setValue(100000000);
    // Set the Yes or No field
    formcontext.getAttribute("demo_yesorno").setValue(true); // or false for No

    if (expenseType == 100000000) {
        // formcontext.getControl("demo_expenseamount").setVisible(true);
        formcontext.getAttribute("demo_expenseamount").setRequiredLevel("required");

        alert("You have selected flight expense");
    } else {
        formcontext.getAttribute("demo_expenseamount").setRequiredLevel("none");
        // formcontext.getControl("demo_expenseamount").setVisible(false);
    }
}
```

For lookup fields it works a little differently. For example, Contact is a lookup field inside Expense Manager Form.

The screenshot shows the Dynamics 365 Expense Manager Form for EXPENSE-100003. The form is titled "EXPENSE-100003 - Saved" and "Expenses - Expense Manager Form". The "General" tab is selected. The form fields are as follows:

Field	Value
Expense ID	EXPENSE-100003
Expense Type	Flight
Currency	Euro
Expense Amount	€300.00
Contact	Avery Howard
Email	---
Yes or No	---
Exchange Rate	1.0000000000
Status Reason	Active
Created By	Nicola Paracone (Offline)
Modified By	Nicola Paracone (Offline)
Owner	Nicola Paracone (Offline)

The "Description" field is empty.

We use a JavaScript script like the following (triggered on change of Contact).

```
function lookupField(executionContext){
    debugger;

    var formcontext = executionContext.getFormContext();

    var contactID = formcontext.getAttribute("demo_contact").getValue();
}
```

Then, once in Dynamics 365, we open the console tool in the browser and add contactID to the watchlist. In this way we will retrieve the Id and name of the contact the expense belongs to.

The screenshot shows the Dynamics 365 Expense Manager Form for "New Expenses" with the "General" tab selected. The form fields are as follows:

Field	Value
Currency	Euro
Expense Amount	---
Contact	Kenya Brady
Email	---
Yes or No	No

The browser's developer console is open, showing the JavaScript script being executed. The script is:

```
function lookupField(executionContext){
    debugger;

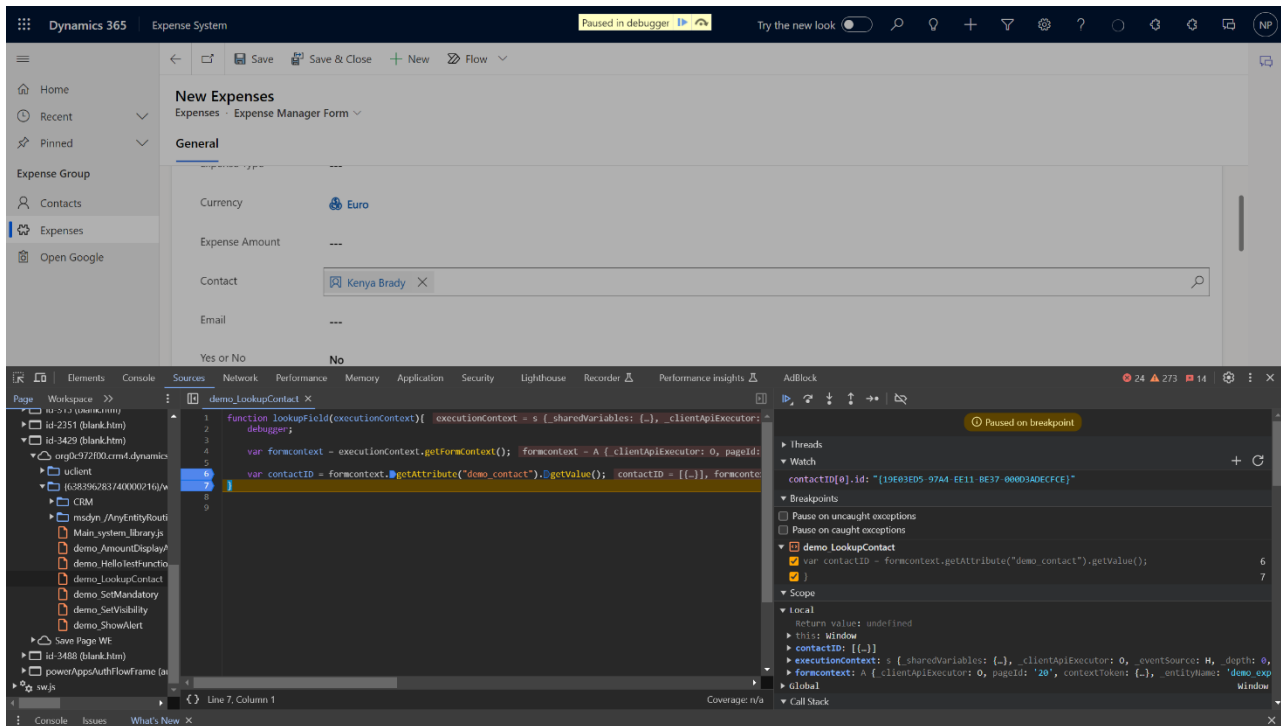
    var formcontext = executionContext.getFormContext();

    var contactID = formcontext.getAttribute("demo_contact").getValue();
}
```

The console shows the script is paused on line 6. The watchlist displays the contact information for Kenya Brady:

```
{
  entityType: "contact",
  id: "48663805-97A4-EE11-B037-000000000000",
  name: "Kenya Brady",
  [[prototype]]: Object,
  length: 1,
  [[prototype]]: Array(0)
}
```

Typing `contactID[0].id` inside the list of variables being watched, we read the actual Guid. `contactID`, in fact, is an array.



2. Hiding and Showing Sections

Navigate to <https://crmminds.com/2022/10/10/dynamics-365-how-to-show-hide-sections-and-form-tabs/>.

Assume that the General tab in the Expense Manager Form has name (not label) `general`, and that the Description section has name `general_description`.

```
function hideShowSections(executionContext) {  
  
    debugger;  
  
    var formcontext = executionContext.getFormContext();  
  
    var expenseType = formcontext.getAttribute("demo_expensetype").getValue();  
  
    // get tab  
    let tabObj = formcontext.ui.tabs.get("general"); // Use the name, not the label!  
    // get section  
    let sectionObj = tabObj.sections.get("general_description"); // Use the name, not the label!  
  
    if (expenseType == 100000000) {  
        // show the tab  
        // tabObj.setVisible(true);  
        // show the section  
        sectionObj.setVisible(true);  
    } else {  
        // hide the tab  
        // hide the section  
        // tabObj.setVisible(false);  
        sectionObj.setVisible(false);  
    }  
}
```

The code above performs the following action: if Expense Type is equal to Flight, then the Description section in the General tab will be visible, otherwise it will be hidden. Always remember that we first have to acquire the tab in which the section resides to act on it; this is because multiple tabs could have sections with the same names. Such operation of showing and hiding sections and/or tabs can be used in relation with the security role assigned with the user navigating on the form.

3. Field and form level notifications

We basically have three kinds of notification: ERROR, WARNING, INFO. It is always important to clear the notification we are setting with `clearFormNotification`, otherwise it will persist on the form. The general syntax is `formContext.ui.setFormNotification(message, level, uniqueId)`. Here we must pass the message we want to be displayed in the notification, level corresponds to one of the possible kinds of notification, and `uniqueId` is any string value identifying the notification.

The code below is triggered on change of Expense Type. If such field has value Flight, then a notification will appear.

```
function simpleNotification(executionContext) {  
  
    debugger;  
  
    var formcontext = executionContext.getFormContext();  
  
    var expenseType = formcontext.getAttribute("demo_expensetype").getValue();  
  
    if (expenseType == 100000000) {  
        formcontext.ui.setFormNotification("You have selected Flight", "INFO", "101");  
    } else {  
        formcontext.ui.clearFormNotification("101");  
    }  
}
```

The screenshot shows the Dynamics 365 Expense System interface. The top navigation bar includes 'Dynamics 365' and 'Expense System'. Below the navigation bar, there's a sidebar with options like Home, Recent, Pinned, Expense Group, Contacts, Expenses, and Open Google. The main area displays the 'New Expenses' form, which is currently in the 'General' tab. The form contains several fields: Expense ID (with a red asterisk), Expense Type (set to 'Flight'), Currency (set to 'Euro'), Expense Amount, Contact, Email, Yes or No (set to 'No'), Exchange Rate, Status Reason (set to 'Active'), Created By, Modified By, and Owner (set to 'Nicola Paracone (Away)'). A notification bar at the top of the form area states 'You have selected Flight'.

The following is a more complex version of the code, potentially displaying one of all the three levels of notification.

```
function formNotify(executionContext) {

    debugger;

    var formcontext = executionContext.getFormContext();

    var expenseType = formcontext.getAttribute("demo_expensetype").getValue();

    if (expenseType == 100000000) {

        formcontext.ui.setFormNotification("You have selected Flight", "INFO", "101");
        var expenseamount = formcontext.getAttribute("demo_expenseamount").getValue();
        if (expenseamount == null) {

            formcontext.ui.setFormNotification("Amount cannot be null", "ERROR", "102");

        } else {

            formcontext.ui.clearFormNotification("102");
            if (expenseamount > 5000) {

                formcontext.ui.setFormNotification("Amount cannot be greater than 5000", "WARNING", "103");

            } else {

                formcontext.ui.clearFormNotification("103");

            }

        }

    } else {

        formcontext.ui.clearFormNotification("101");

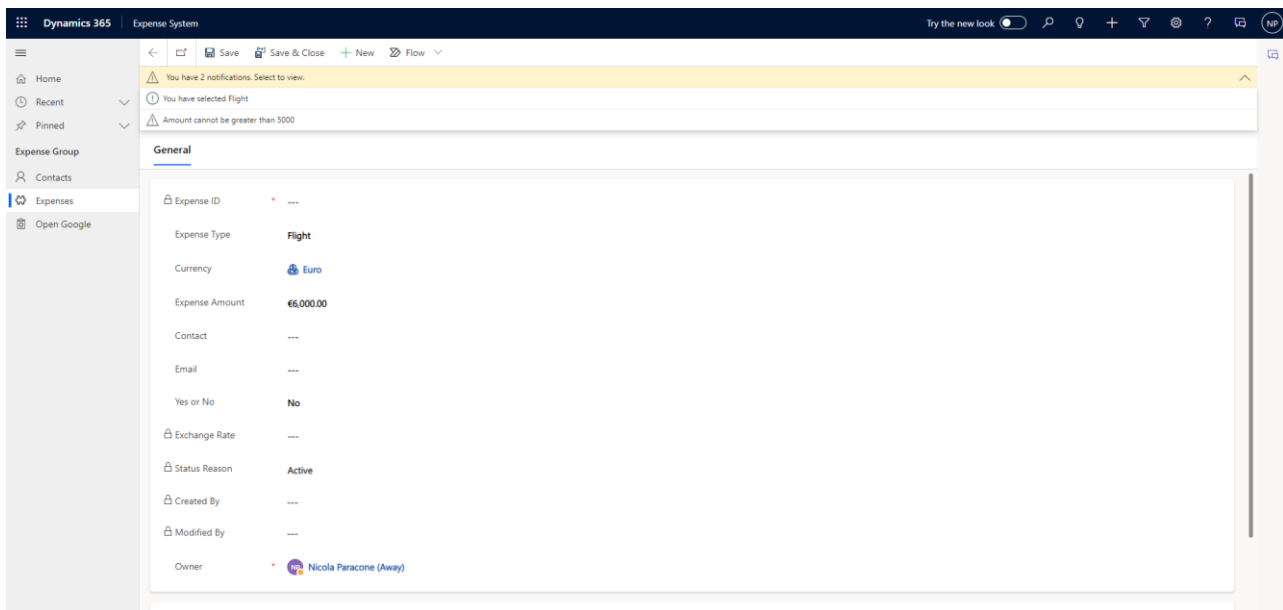
    }

}
```

These are examples of what we could potentially observe in Dynamics 365.

The screenshot displays the Dynamics 365 Expense System interface. At the top, a notification bar indicates "You have 2 notifications. Select to view." Below this, the "General" tab is active, showing a form with the following fields:

Field	Value
Expense ID	---
Expense Type	Flight
Currency	Euro
Expense Amount	---
Contact	---
Email	---
Yes or No	No
Exchange Rate	---
Status Reason	Active
Created By	---
Modified By	---
Owner	Nicola Paracone (Away)



For field level notifications, navigate to <https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/reference/controls/addnotification> to have a reference and some examples. Check in particular the following one.

```
function addTickerSymbolRecommendation(executionContext) {
    var formContext = executionContext.getFormContext();
    var myControl = formContext.getControl('name');
    var accountName = formContext.data.entity.attributes.get('name');
    var tickerSymbol = formContext.data.entity.attributes.get('tickersymbol');

    if (accountName.getValue() == 'Microsoft' && tickerSymbol.getValue() != 'MSFT') {
        var actionCollection = {
            message: 'Set the Ticker Symbol to MSFT?',
            actions: null
        };

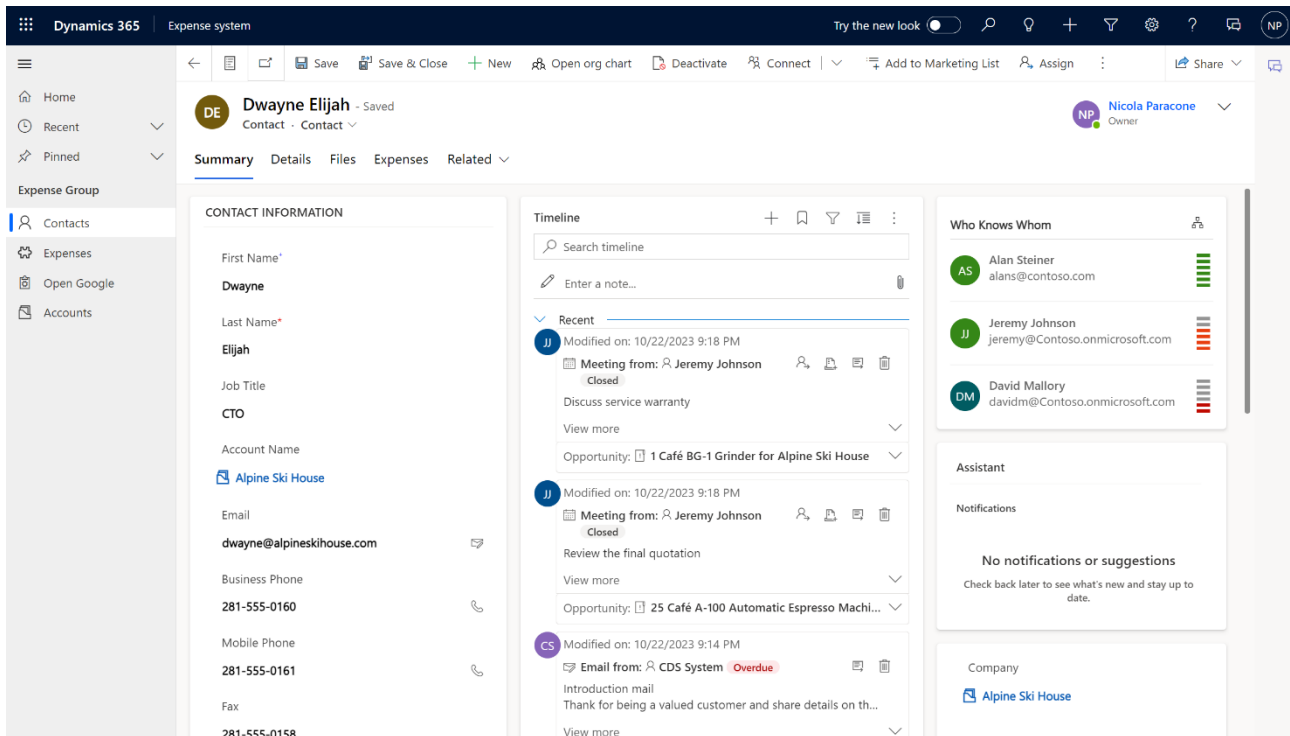
        actionCollection.actions = [function () {
            tickerSymbol.setValue('MSFT');
            myControl.clearNotification('my_unique_id');
        }];

        myControl.addNotification({
            messages: ['Set Ticker Symbol'],
            notificationLevel: 'RECOMMENDATION',
            uniqueId: 'my_unique_id',
            actions: [actionCollection]
        });
    }
    else
        console.log("Notification not set");
}
```

Navigate to <https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/reference/formcontext-ui/setformnotification> for more instructions and an in-depth explanation.

4. Set a lookup field

One more thing to be learnt is how to set the value of a lookup field. We recall that Contact is a lookup field in Expenses. Suppose that the contact we want to set is the following.



This has a global unique identifier which can be retrieved in the URL.

```
// To set a lookup field
var lookupData = new Array(); // Create a lookup data array
var lookupItem = new Object(); // Create a lookup item
lookupItem.id = "9fd4a450-cb0c-ea11-a813-000d3a1b1223";
lookupItem.name = "Dwayne Elijah";
lookupItem.entityType = "contact"; // This is the logical name of the Contact entity
lookupData[0] = lookupItem;
formcontext.getAttribute("demo_contact").setValue(lookupData);
```

So, if we want to set the Contact field to Dwayne Elijah, first we create an empty array of lookup data. Then we create an object with keys id, name and entityType. Notice that id corresponds precisely to the Guid associated with Dwayne Elijah, and that entityType contains the logical name of the Contact entity (which is a table). The first element of the newly created array is to be set equal to the object we have defined. Finally, we set the value of the Contact field in Expenses, which has logical name demo_contact, with the lookup item.